

Supplementary Material: Reading the Latest, Not the Loudest: Lifecycle-Aware Structured Memory for Retrieval-Augmented Generation

Anonymous submission

This supplement collects auxiliary figures and the full reproducibility map for the main paper. All numbers are produced by the released JSONs; figure and table identifiers below carry an “S” prefix and do not renumber the main paper.

A Quantitative error localization

Proposition 1 makes all detector assumptions explicit. Here is its quantitative counterpart under correct temporal order. Let $M(a, b)$ be SAMESUBJECTRELATION and $V(a, b)$ be CHANGEDVALUE. A later claim c' is a *true witness* for c when $\kappa(c') = \kappa(c)$ and $v_{c'} \neq v_c$; it is a *detected witness* when $M(c', c) \wedge V(c', c)$. Let FW count claims outside S^* that have a detected witness, and let MW count claims in S^* for which no true witness is detected.

Proposition 1 (Error localization, quantitative). *The detector’s superseded set $\hat{S}$ satisfies $|\hat{S} \setminus S^*| \leq \text{FW}$ and $|S^* \setminus \hat{S}| \leq \text{MW}$, so precision is at least $1 - \text{FW}/|\hat{S}|$ and recall at least $1 - \text{MW}/|S^*|$. When both counts are zero, $\hat{S} = S^*$.*

Proof. The detector marks c iff it has a detected witness. Thus each $c \in \hat{S} \setminus S^*$ is counted by FW, and each $c \in S^* \setminus \hat{S}$ by MW; division gives the bounds. $\square$

Detected-witness errors can come from a key merge/split by M or a false/missed value change by V ; wrong or missing timestamps violate the proposition’s order premise. In the audited RealProse errors, key parsing and alias matching dominate, while the value and timestamp ablations test the other two routes separately.

B Text-driven temporal-KG baseline (exact key)

The historical as-of comparison in the main paper includes a temporal knowledge-graph baseline built from the same text signal as our detector, differing only in the matcher. It extracts each claim’s frame and subject phrases exactly as the detector does, but groups claims into one timeline only when their

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

(frame, phrases) keys are *identical*, then chains each group in timestamp order into validity intervals. With clean verbatim keys it recovers the timeline (Real-world reaches 1.000), but paraphrase and co-reference split one entity across components: historical top-1 accuracy is 0.925 on Wikidata and 0.933 on TempLAMA, versus 0.940 and 0.973 for the fuzzy lifecycle matcher. The exact-key baseline uses no labels and is deterministic; it isolates the matcher, rather than the interval construction, as the source of the difference. Full results are in the three released `results/asof*` cards.

Direct temporal-RAG head-to-head. TempRALM (Gade and Jetcheva 2024) and RAG-Time (Grofsky 2025) receive the same corpus, question, five-claim budget, prompt, reader, and scorer as lifecycle; only retrieval changes. TempRALM adds a distribution-matched reciprocal time-distance score to raw dense cosine similarity. RAG-Time scores $0.7s + 0.3(0.5)^{a/1825}$, where s is dense cosine and a age in days. This setting is selected from 20 settings on 9 Wikidata development keys by mean current/as-of accuracy, then frozen (`results/ragtime_heldout_tuning.json`). The main 33-key Wikidata aggregate includes development keys; TempLAMA is fully external.

TempRALM’s weak current-answer row is not a sort-direction artifact: its full-pool temporal term sacrifices topicality. Under the exact top-five protocol, the mean number of requested-key claims falls from 4.000 to 1.121 on Wikidata and from 2.460 to 0.643 on TempLAMA; active-claim inclusion falls from 0.879 to 0.273 and from 0.993 to 0.350, respectively. The deterministic audit is released as `results/tempralm_topicality_audit.json`. RAG-Time’s held-out tuning instead raises its current-answer EM to 0.697/0.546 on Wikidata and 0.657/0.473 on TempLAMA (extractive/Qwen), while lifecycle remains 0.970/0.970 and 0.953/0.937.

C Reproducible real-prose benchmarks and parser robustness

The main-paper Real-Wikipedia set (F1 0.880) is the detector’s honest single-relation test on genuine encyclopedic prose. To test whether the zero-LLM detector *generalizes* beyond that relation, we release two in-repo benchmarks of unedited English Wikipedia leads, with gold taken from known chronology *order* rather than any parse of ours.

Benchmark	Span text visible to detector	Time visible to detector	Gold tuple source (scoring only)
FreshRAG	generated policy-circular template	generated month/year	synthetic scheme, metric, value
Real-world	sentence rendered from curated change	curated year	hand-curated subject, relation, value
Wikidata	REST labels rendered in property template	start-time qualifier	Wikidata subject/property/value IDs
TempLAMA	published query template filled with answer	benchmark year	subject QID, relation PID, answer
Wikidata-prose	seeded paragraph paraphrase with distractor	inherited Wikidata time	inherited Wikidata tuple
RealWiki	Wikipedia CEO lead extract	known tenure start	curated company, role, executive
RealProse	Wikipedia lead extract	known chronology start	curated subject, role, officeholder

Table S1: **Input provenance.** Structured annotations construct data and define gold; Algorithm 1 receives only the rendered or extracted span and its timestamp. RealWiki and RealProse are the genuine-prose parsing tests.

RealWiki (CEO leads). 26 CEO lead extracts, one claim per executive, are chained into 12 company chronologies. These are genuine leads, with biographical clauses, pronoun coreference, and several competing proper nouns per sentence. Two lightweight, zero-LLM parser rules handle common noise. (1) *Demonym filtering*: nationality adjectives (“American”, “Indian-American”) are dropped as subject phrases, since a shared “American” could otherwise cross-link unrelated companies. (2) *Role-acronym normalization*: “CEO” and kin are treated as relation words rather than proper nouns, so “CEO of Apple” parses as (relation, subject Apple). The released card reports F1 0.880 at precision 1.000 and recall 0.786 (results/detection_realwiki.json).

RealProse (five relation families). To probe generalization at scale we release REALPROSE: 132 genuine Wikipedia leads over 35 subjects and five relation families – company chief executives, national heads of government, national heads of state, football-club managers, and international-organization leaders. All claims share one pooled index, so same-relation pairs with a *different* subject are hard negatives the detector must not cross-link. The same zero-LLM detector reaches F1 0.941 at precision 0.978 and recall 0.907, with two false positives across the entire pooled index. Spelled-out office titles (“President”, “Chancellor”, “Prime Minister”) are treated as relation words rather than subject phrases, keeping the subject representation stable across capitalization variants (results/detection_realprose.json).

The audited residual errors are key-matcher failures. Recall misses are biographies that dilute the relation frame below the 0.5 overlap gate, corporate-name variants, an entity renamed mid-tenure, and leads that append a second office by “and”. Both false positives are football managers sharing “Premier League” while one lead omits the club. These observations do not exclude changed-value or timestamp failures in other corpora; those routes are tested separately.

How often long histories occur. The lifecycle gain concentrates on subjects that carry many superseded values, so its practical reach depends on history length (reproducible via experiments/history_length_stats.py). Genuine Wikipedia leads average 3.7 values per subject with 49% holding four or more (RealProse), and the hand-curated real-world set averages 2.8. TempLAMA’s fixed-decade snapshots leave 69% of subjects with two values and 31 with four to eight; on that complete long-history slice lifecycle reaches

1.000 EM versus 0.710 for flat retrieval.

D Third-party current-answer benchmark (TempLAMA-QA)

The third-party current-answer experiment is built from the published TempLAMA test split with no labels of ours. Rows are grouped by subject and relation, consecutive identical answers are collapsed into value transitions, and groups with fewer than two distinct values are dropped, yielding 300 chronologies over 9 relations with 447 gold-superseded claims; the gold supersession labels follow directly from TempLAMA’s year annotations. Each transition is rendered as a *plain present-tense* sentence (e.g. “Clemens Fuest works for University of Mannheim.”), so recency lives only in document metadata rather than leaking through an in-sentence year. The canonical study asks one current-answer question for every chronology and applies Flat RAG, TempRALM, RAG-Time, lifecycle retrieval, and the gold-edge oracle with the same five-claim budget, prompt, extractive or Qwen2.5-3B-Instruct reader, and EM scoring used for Wikidata. There is no paraphrase pooling or reduced subset: the main-paper table reports all 300 paired questions per reader, directly from results/qa_templama_extractive.json and results/qa_templama_qwen.json.

E Long histories: a wide margin on third-party data

History length provides a mechanism check within the same third-party data: 207 of the 300 TempLAMA chronologies hold only two values, whereas 31 contain four to eight. We therefore repeat the current-answer comparison on that complete long-history slice, dominated by football head-coach successions (P286) and player-club changes (P54), with a few multi-post politicians (P39). Data, questions, and labels remain TempLAMA’s.

Each chronology’s competing present-tense values are shown to a local Qwen2.5-3B-Instruct reader under four conditions: *flat*, *date-prompt*, *date-rerank*, and *lifecycle*. Table S2 reports the result: on the full long-history slice lifecycle reaches 1.000 EM, versus 0.710 for flat retrieval, 0.645 for date prompting, and 0.290 for newest-first ordering (all paired $p < 0.001$, $n=31$). The deterministic first-item reader scores newest-first and lifecycle at 1.000, confirming the sort direction; Qwen’s

reversal is a reader position bias, not reversed dates (results/history_templama_*.json).

Condition	truncated to m				full
	$m=2$	$m=3$	$m=4$	$m=5$	$m \geq 4$
N keys	40	40	31	15	31
flat	0.975	0.850	0.774	0.800	0.710
date-prompt	0.975	0.600	0.677	0.600	0.645
date-rerank	0.525	0.500	0.323	0.333	0.290
lifecycle	1.000	1.000	1.000	1.000	1.000

Table S2: **Long histories, third-party TempLAMA** (Qwen2.5-3B reader, current-value EM). Per- m columns truncate each chronology to its latest m values; the final column is the full long-history slice ($m \geq 4$, mean 4.74 values). Source: results/history_templama_qwen.json.

F Realistic prose: how much of the gain survives recency cues

The main-paper Wikidata experiment renders claims as plain present-tense statements with recency only in document metadata. A fair question is how much survives when text carries natural recency cues. We answer it end to end on RealProse genuine Wikipedia leads (Section C). For each subject with a real change we ask a current-role question (35 questions), retrieve over all RealProse claims with the same hybrid retriever, and read with local Qwen2.5-3B; the extractive reader is a cross-check.

Two effects appear (Table S3). Natural cues lift flat Qwen EM to 0.714, compared with 0.424 on Wikidata. Lifecycle still raises RealProse Qwen EM to 0.857 (gold 0.914), a gain of 0.143 (95% CI [0.000, 0.286], $p=0.076$); the extractive gain is 0.343 ($p < 0.001$). Retrieval quality improves independently of reader: SER falls from 0.971 to 0.229 and AEP rises from 0.319 to 0.859. Global date reranking is weakest (0.143 EM) because it can surface an unrelated recent appointment. The evidence therefore supports a narrower statement: prose cues help Qwen resolve some conflicts, while lifecycle still removes stale context and moves both readers toward the oracle (results/qa_realprose_qwen.json, results/qa_realprose_extractive.json).

Condition	Qwen EM	extractive EM
flat	0.714	0.543
date-prompt	0.686	0.543
date-rerank	0.143	0.143
lifecycle	0.857	0.886
gold-edge oracle	0.914	0.914
SER (flat $\rightarrow$ lifecycle)	0.971 $\rightarrow$ 0.229	
AEP (flat $\rightarrow$ lifecycle)	0.319 $\rightarrow$ 0.859	

Table S3: **Realistic end-to-end current-answer QA** on 35 genuine-prose questions (RealProse leads, hybrid retrieval). Source: results/qa_realprose_*.json.

G Open-weights reader and strict scoring

Two robustness checks confirm the current-answer gains are neither a closed-model nor a lenient-metric artifact. First, an *open-weights* reader reproduces the effect: with greedy decoding, Qwen2.5-3B-Instruct answers 0.424 under flat retrieval, 0.515 with a trust-the-latest-date prompt, and 0.364 after date reranking on the controlled Wikidata current-fact set, while lifecycle retrieval reaches 0.970 and matches the gold-edge oracle (0.970); the same fifty-point gap over every date-only trick appears with no proprietary reader (results/qa_wikidata_qwen.json). Second, we re-score every condition under a stricter *clean* exact match that counts a hit only when the answer contains the gold value *and none* of the key’s superseded values, so an answer that hedges by naming an outdated value too is rejected. Clean EM equals the lenient EM for both readers on every condition (current_answer_em_strict in results/qa_wikidata_extractive.json and results/qa_wikidata_qwen.json): the readers return single-valued answers, so the reported margins do not depend on lenient substring scoring.

H Adversarial cheap-baseline audit

Every headline in the paper was re-tested against the strongest *cheap* competitor we could construct, on the principle that a number is only publishable if a trivial trick cannot reproduce it. This section reports all of them, including the ones that tie us.

Detection: newest-document-wins. The zero-LLM competitor marks every claim outside a case’s newest document superseded. Both methods are strong when cases are isolated, but newest-document-wins cannot distinguish stable facts once subjects share an index. Table S4 therefore reports isolated and pooled evaluation over the same four structured corpora.

Method	Isolated F1	Pooled P	Pooled R	Pooled F1
Lifecycle heuristic	0.991	0.986	0.984	0.985
Newest-document-wins	0.985	0.608	1.000	0.756

Table S4: Both methods are strong in isolation, but pooling 1,315 claims exposes false supersessions from newest-document-wins. Source: the released four-corpus pooled card.

Detection: a stronger LLM judge. On the identical 240 Wikidata span pairs, gpt-4o-mini reaches F1 0.960 and gpt-4.1-mini reaches 0.992, while the zero-LLM structural rule remains exact at 1.000. The two released results/judge_wikidata* cards report full precision and recall.

QA: dates in the prompt. Every packed snippet is date-stamped. Under the canonical five-claim protocol, the extractive reader reaches 0.182 with an explicit date prompt and 0.121 with date reranking on Wikidata, versus 0.970 for lifecycle; Qwen reaches 0.515,

0.364, and 0.970. On TempLAMA, the extractive reader reaches 0.453 with a date prompt and 0.127 with date reranking, versus 0.953 for lifecycle. Every lifecycle gap is significant at $p < 0.001$ in the paired result cards (`results/qa_wikidata_extractive.json`, `results/qa_wikidata_qwen.json`, `results/qa_templama_extractive.json`).

QA: history length isolates the mechanism. The main-paper table truncates Wikidata chronologies to $m \in \{2, 3, 4, 5\}$ values, yielding 33, 29, 22, and 15 Qwen-evaluated keys. Flat AEP is exactly $1/m$; lifecycle EM remains 1.000, while flat, date-prompt, and date-rerank EM vary as stale values accumulate. Every displayed value comes from `results/history_sweep.json`.

Detection at index scale. We expand the 1,315-claim pooled corpus with unique-subject distractors to 2,630, 3,945, and 6,575 claims. Precision, recall, and F1 remain 0.986, 0.984, and 0.985 because distractors add no shared key. Blocking comparisons by extracted key returns the identical edge set as all-pairs comparison at every tested size (`results/scaling.json`); wall-clock time is machine- and load-dependent and is not used as an effectiveness claim.

I Grounding scope

RealWiki and RealProse store source-attributed Wikipedia lead extracts; structured benchmarks use the documented renderings in Table S1. The detector reads only span text and time in either case. This does not guarantee correct span construction, timestamps, value comparison, or lifecycle links. Detection, timestamp-noise, and genuine-prose experiments evaluate those routes separately; lifecycle retrieval filters predicted-stale spans before top- k .

J Reproducibility and transparency bundle

We release a single archive with the source code, datasets, result JSONs, paper table generator, and the on-disk Qwen generation cache, so the open-reader results replay without an API key. `results/MANIFEST.json` records a SHA-256 digest for every dataset and result card plus an at-a-glance map of headline results. Running `python -m experiments.build_bundle -verify` re-hashes the archive and must print ALL VERIFIED; `python scripts/make_paper_tables.py` regenerates the paper tables directly from those cards.

Execution environment. Experiments were run on Microsoft Windows 11 Enterprise (build 10.0.26200) with a 13th-generation Intel Core i7-13800H CPU, 63.8 GB RAM, and an NVIDIA RTX 2000 Ada Generation Laptop GPU with 8,188 MiB VRAM (driver 591.55). The released environment uses Python 3.11.15, NumPy 2.4.6, PyTorch 2.6.0 with CUDA 12.4, Transformers 5.12.1, Sentence-Transformers 5.6.0, scikit-learn 1.9.0, NetworkX 3.6.1, and rank-bm25 0.2.2; `requirements.txt` pins the complete environment. Qwen runs locally in fp16 on the single GPU. Detector, retrieval, extractive-reader, and greedy-Qwen conditions are deterministic and run once. Randomized ablations

use seeds 0–4; timestamp perturbations use five deterministic seeds per nonzero noise level. Confidence intervals and paired tests use 2,000 bootstrap resamples.

Final settings. Hybrid retrieval equally weights min–max-normalized MiniLM cosine and BM25 scores ($\alpha=0.5$), uses exact matrix–vector cosine scoring, and returns five claims. The detector uses frame-overlap threshold 0.5 and edge confidence 0.9 when an update cue is present (0.7 otherwise). Qwen uses greedy decoding with at most 24 new tokens. Development selection gives RAG-Time $\alpha=0.7$ and a 1,825-day half-life; TempRALM uses reciprocal temporal distance normalized to the dense score distribution.

References

- Gade, A.; and Jetcheva, J. 2024. It’s About Time: Incorporating Temporality in Retrieval Augmented Language Models. *arXiv preprint arXiv:2401.13222*.
- Grofsky, M. 2025. Freshness and the Limits of Heuristic Trend Detection in Temporal RAG. *arXiv preprint arXiv:2509.19376*.